TEORIA DOS GRAFOS

Material de apoio

Davi Valdez

Gabriel Bielawski

Gustavo Almeida

Kauê Barbi

Larissa Ibiapino

Sumário

Unidade 1: Definição de Grafos. Vértices e Arestas

Unidade 2: Funções Sobrejetora, Injetora e Bijetora

Unidade 3: Handshake Lemma

Unidade 4: Planaridade dos Grafos - Isomorfismo - Teorema de Kuratowski

Unidade 5: Matriz de adjacência e Lista de adjacência

Unidade 6: Percurso em Profundidade (DFS) e Percurso em Largura (BFS)

Unidade 7: Algoritmo de Dijkstra e Bellman-Ford

Unidade 8: Algoritmo de Prim

Unidade 1

Definição de grafos, vértices e arestas

1.1 Por que estudar grafos?

Um grafo é uma estrutura matemática usada para representar objetos e as relações entre eles. No mundo da tecnologia, essa abstração aparece em redes sociais, rotas de transporte, redes de computadores, dependências entre tarefas, links entre páginas, circuitos, sistemas de recomendação e inúmeras outras situações.

> **Pense assim**
> Os vértices representam “coisas”. As arestas representam “relações” entre essas coisas. A força dos grafos está em ignorar detalhes irrelevantes e preservar apenas a estrutura relacional importante para o problema.

1.2 Definição formal

Um grafo simples não direcionado pode ser representado por um par ordenado G = (V, E), em que V é um conjunto não vazio de vértices e E é um conjunto de arestas. Cada aresta associa dois vértices distintos (ROSEN, 2019; WEST, 2001).

G = (V, E)

V: conjunto de vértices • E: conjunto de arestas

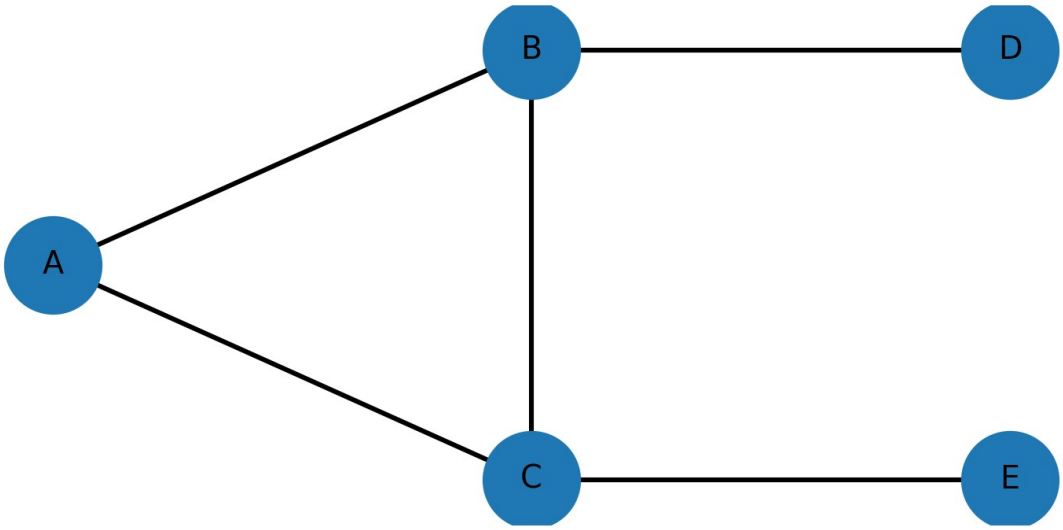


Figura 1 — Exemplo de grafo simples com cinco vértices.

Elemento	Representação no exemplo	Interpretação
Conjunto de vértices	V = {A, B, C, D, E}	Objetos presentes no sistema

Conjunto de arestas	E = {{A,B},{A,C},{B,C},{B,D}, {C,E}}	Relações existentes entre pares de vértices
Ordem do grafo	|V| = 5	Quantidade de vértices
Tamanho do grafo	|E| = 5	Quantidade de arestas

1.3 Vértices e arestas na modelagem

A mesma estrutura pode representar problemas totalmente diferentes. O significado de um vértice depende do contexto. Por exemplo, se os vértices forem pessoas, as arestas podem significar amizade; se forem cidades, as arestas podem significar estradas; se forem módulos de software, as arestas podem significar dependência.

Problema	**Vértices**	**Arestas**
Rede social	Usuários	Relações de amizade
Malha rodoviária	Cidades	Estradas diretas
Rede de computadores	Dispositivos	Conexões de comunicação
Dependência de tarefas	Tarefas	Relações de precedência/dependência

1.4 Grafo direcionado e não direcionado

Quando a relação é simétrica, costuma-se usar um grafo não direcionado. Uma amizade, por exemplo, pode ser representada por uma aresta sem orientação. Quando a relação possui sentido, usa-se um grafo direcionado: “A segue B” não implica necessariamente que “B segue A”.

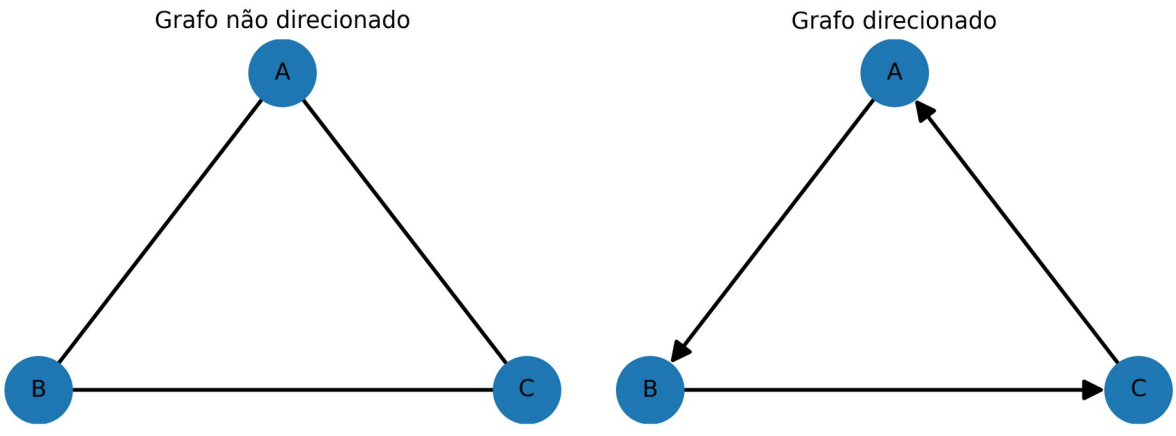


Figura 2 — Comparação entre arestas sem direção e arcos direcionados.

Ponto de atenção

Em um grafo direcionado, a ordem importa: (u,v) e (v,u) podem representar relações diferentes. Em um grafo simples não direcionado, {u,v} representa a mesma aresta que {v,u}.

1.5 Adjacência, incidência e vizinhança

- Dois vértices são adjacentes quando existe uma aresta ligando-os diretamente.
- Uma aresta é incidente aos vértices que aparecem em suas extremidades.
- A vizinhança N(v) de um vértice v é o conjunto de vértices adjacentes a v.
- Em um grafo simples, não há laços e não há arestas paralelas entre o mesmo par de vértices.

No grafo da Figura 1, B é adjacente a A, C e D. Portanto, N(B) = {A, C, D}. A aresta {B,D} é incidente tanto em B quanto em D.

1.6 Exemplo de abstração computacional

Considere quatro servidores S1, S2, S3 e S4. Há conexões diretas entre S1–S2, S1–S3, S2–S3 e S3–S4. O modelo discreto correspondente é:

$$V = \{S1, S2, S3, S4\} \quad E = \{\{S1,S2\}, \{S1,S3\}, \{S2,S3\}, \{S3,S4\}\}$$

Esse modelo permite responder perguntas sem conhecer cabos, fabricantes ou distância física: S4 possui apenas uma conexão direta; S1 e S2 compartilham um vizinho; e S3 atua como ponto de conexão para todos os demais.

Síntese da Unidade 1

Um grafo preserva relações. Identificar corretamente o conjunto V e o conjunto E é o primeiro passo para qualquer análise posterior. Antes de calcular, sempre declare o que os vértices e as arestas significam no problema.

Unidade 2

Funções Sobrejetora, Injetora e Bijetora

2.1 Função como correspondência entre conjuntos

Uma função f: A → B associa cada elemento do conjunto A, chamado domínio, a exatamente um elemento do conjunto B, chamado contradomínio. A classificação da função depende de como os elementos do domínio atingem o contradomínio (ROSEN, 2019).

$$f : A \rightarrow B$$

- Domínio: conjunto de onde partem os elementos.
- Contradomínio: conjunto de possíveis valores de chegada.
- Imagem: subconjunto do contradomínio que é efetivamente atingido pela função.

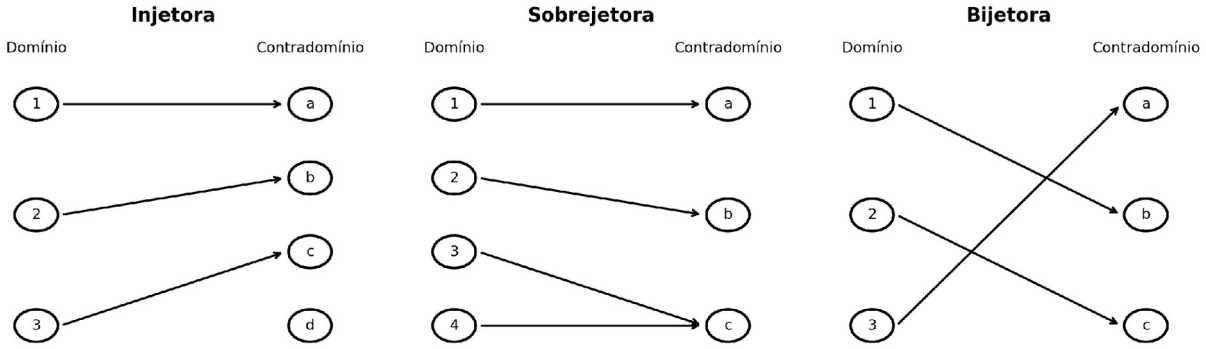


Figura 3 — Comparação visual entre funções injetora, sobrejetora e bijetora.

2.2 Função injetora

Uma função é injetora quando elementos diferentes do domínio possuem imagens diferentes. Em outras palavras, dois elementos distintos de A não podem "colidir" no mesmo elemento de B.

$$f(x_1) = f(x_2) \Rightarrow x_1 = x_2$$

Forma lógica equivalente à injetividade.

Leitura visual

Na injeção, pode sobrar elemento no contradomínio. O que não pode ocorrer é dois elementos distintos do domínio apontarem para a mesma imagem.

Exemplo: A={1,2,3}, B={a,b,c,d} e f(1)=a, f(2)=b, f(3)=c. A função é injetora, pois nenhuma imagem se repete. Entretanto, não é sobrejetora porque d não é atingido.

2.3 Função sobrejetora

Uma função é sobrejetora quando todo elemento do contradomínio é imagem de pelo menos um elemento do domínio. Assim, a imagem da função coincide com todo o contradomínio.

$$\forall\, y \in B, \exists\, x \in A \text{ tal que } f(x) = y$$

> **Leitura visual**
> Na sobrejeção, nenhum elemento do contradomínio fica sem receber seta. É permitido que dois ou mais elementos do domínio cheguem ao mesmo elemento de B.

Exemplo: A={1,2,3,4}, B={a,b,c}, com f(1)=a, f(2)=b, f(3)=c e f(4)=c. Todo elemento de B é atingido, logo a função é sobrejetora. Ela não é injetora porque 3 e 4 possuem a mesma imagem.

2.4 Função bijetora

Uma função é bijetora quando é simultaneamente injetora e sobrejetora. Isso significa que existe correspondência um a um entre os elementos do domínio e do contradomínio: ninguém "colide" e ninguém "fica de fora".

Bijetora = Injetora + Sobrejetora

Se os conjuntos são finitos, uma bijeção entre A e B implica $|A| = |B|$. Além disso, toda função bijetora possui função inversa $f^{-1}: B \to A$.

Tipo	Pode repetir imagem?	Pode sobrar elemento em B?	Ideia central
Injetora	Não	Sim	Cada entrada possui imagem exclusiva
Sobrejetora	Sim	Não	Todo elemento do contradomínio é atingido
Bijetora	Não	Não	Correspondência

			perfeita um a um

2.5 Como testar uma função

1. Confirme se cada elemento do domínio possui exatamente uma imagem.
2. Procure colisões: se dois elementos distintos chegam ao mesmo ponto, não é injetora.
3. Procure elementos não atingidos no contradomínio: se existir algum, não é sobrejetora.
4. Se passar simultaneamente pelos testes de injeção e sobrejeção, a função é bijetora.

2.6 Relação com grafos e isomorfismo

Em teoria dos grafos, a ideia de bijeção é essencial para definir isomorfismo. Dois grafos são isomorfos quando existe uma bijeção entre seus conjuntos de vértices que preserva adjacências. Isso formaliza a ideia de que os grafos têm a mesma estrutura, ainda que estejam desenhados de forma diferente ou usem nomes diferentes para seus vértices (BONDY; MURTY, 2008).

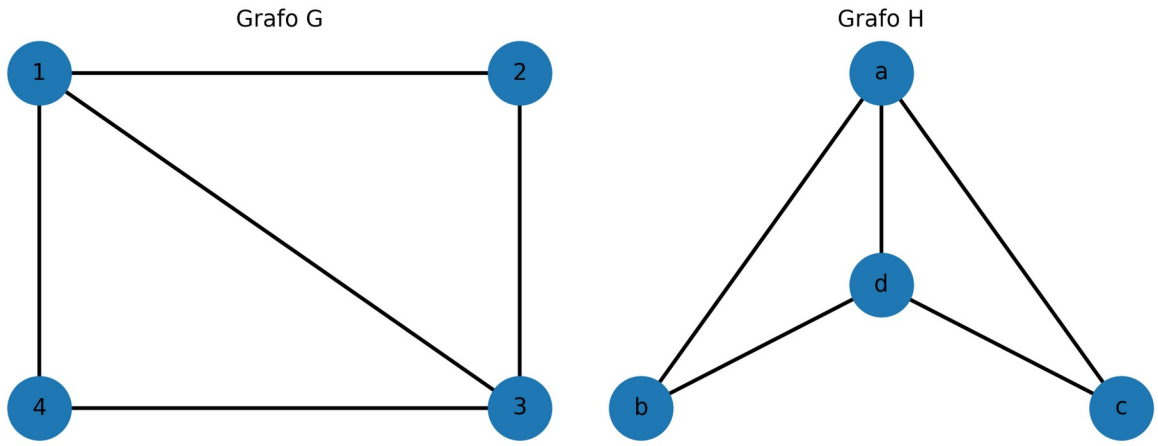


Figura 4 — Dois desenhos diferentes podem representar a mesma estrutura de adjacências.

No exemplo, uma correspondência possível é 1↦a, 2↦b, 3↦d e 4↦c. Observe que cada aresta de G corresponde a uma aresta em H sob essa associação. Não basta ter o mesmo número de vértices e arestas: a bijeção precisa preservar a estrutura.

Síntese da Unidade 2

Injeção controla repetição de imagens; sobrejeção controla cobertura do contradomínio; bijeção satisfaz as duas condições. Em grafos, a bijeção é a base para comparar estruturas e

formalizar isomorfismos.

Unidade 3

Handshake Lemma

3.1 Antes do lema: grau de um vértice

Em um grafo não direcionado, o grau deg(v) de um vértice v é a quantidade de arestas incidentes a ele. O grau mede quantas conexões diretas aquele vértice possui.

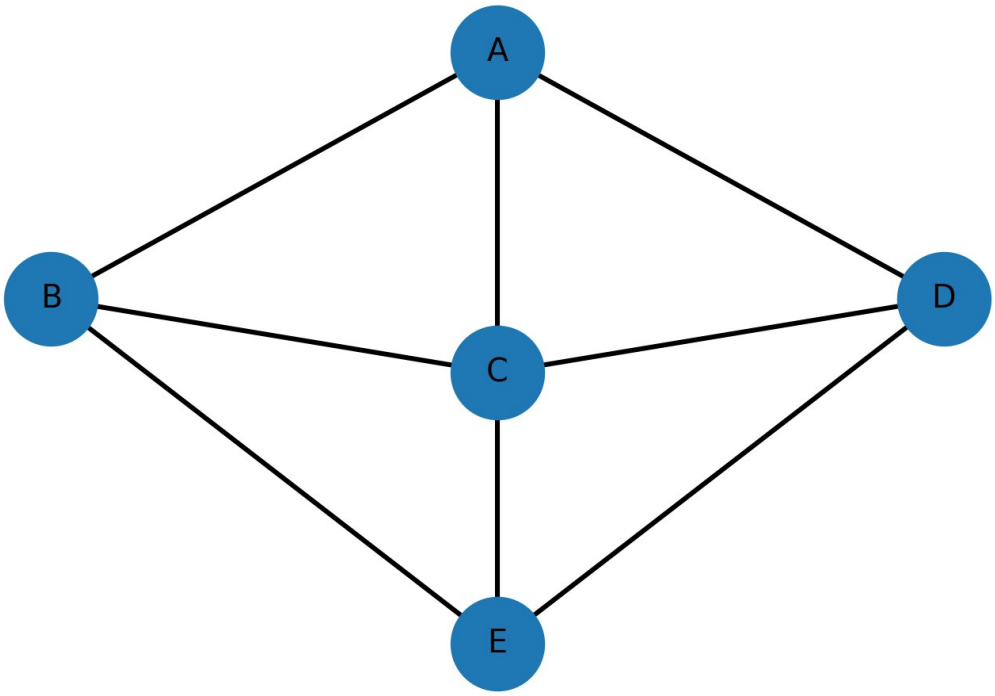


Figura 5 — Grafo usado para analisar graus e o Handshake Lemma.

Vértice	Vizinhos	Grau
A	B, C, D	3
B	A, C, E	3
C	A, B, D, E	4
D	A, C, E	3
E	B, C, D	3

Somando os graus: 3 + 3 + 4 + 3 + 3 = 16. O grafo possui 8 arestas. Observe que 16 = 2 × 8. Essa regularidade não é coincidência.

3.2 Enunciado do Handshake Lemma

O Handshake Lemma, também conhecido como Lema do Aperto de Mãos, afirma que, em qualquer grafo finito não direcionado, a soma dos graus de todos os vértices é igual ao dobro do número de arestas (ROSEN, 2019).

$$\Sigma_{v \in V} \deg(v) = 2|E|$$

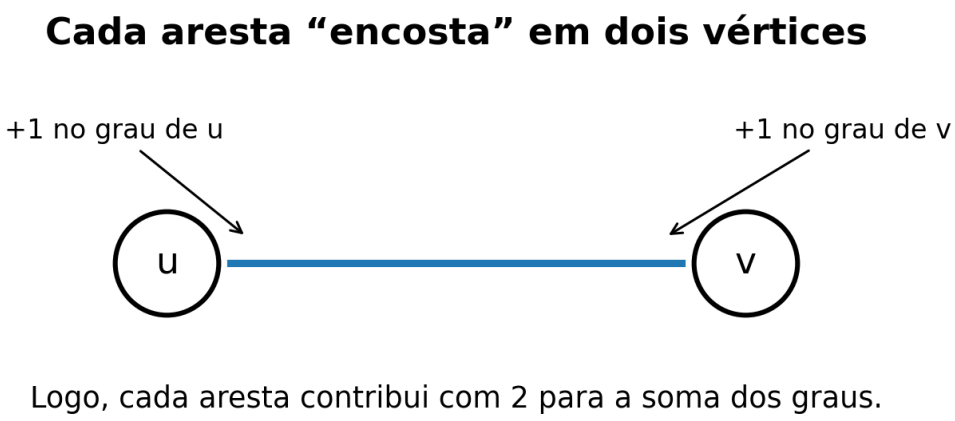


Figura 6 — Intuição por trás do fator 2 no Handshake Lemma.

3.3 Por que o lema é verdadeiro?

A prova usa uma contagem dupla. Imagine contar, para cada vértice, quantas extremidades de arestas chegam até ele. Cada aresta possui exatamente duas extremidades. Portanto, ao somar os graus de todos os vértices, cada aresta é contabilizada duas vezes: uma em cada ponta.

1. Considere todas as arestas do grafo.
2. Cada aresta contribui com 1 para o grau de cada uma de suas duas extremidades.
3. Logo, cada aresta contribui com 2 unidades para a soma total dos graus.
4. Com $|E|$ arestas, a contribuição total é exatamente $2|E|$.

> **Rigor importante**
>
> O lema não diz que cada vértice possui grau par. Ele diz que a SOMA dos graus é par, pois é igual a duas vezes o número de arestas.

3.4 Exemplo resolvido

Suponha que uma rede simples tenha seis vértices com graus 3, 2, 4, 1, 2 e 4. Quantas arestas existem?

$$\Sigma \deg(v) = 3 + 2 + 4 + 1 + 2 + 4 = 16$$

$$16 = 2|E| \Rightarrow |E| = 8$$

O grafo possui 8 arestas. Note que o lema permite determinar o número de arestas sem precisar visualizar ou enumerar uma a uma todas as conexões.

3.5 Consequência: número de vértices de grau ímpar

Como a soma de todos os graus é par, a quantidade de vértices com grau ímpar também deve ser par. Isso ocorre porque uma soma de números inteiros é par somente quando a quantidade de parcelas ímpares é par.

> **Consequência clássica**
>
> Em qualquer grafo finito não direcionado, o número de vértices de grau ímpar é par.
>
> Portanto, não pode existir um grafo com exatamente 1, 3, 5,... vértices de grau ímpar.

3.6 Exemplo de diagnóstico

Uma pessoa afirma ter construído um grafo simples com cinco vértices cujos graus são 3, 3, 2, 2 e 1. Antes mesmo de desenhar o grafo, podemos testar a consistência da sequência:

$$3 + 3 + 2 + 2 + 1 = 11$$

Como 11 é ímpar, essa sequência não pode ser a sequência de graus de um grafo não direcionado. O Handshake Lemma exige que a soma dos graus seja igual a $2|E|$ e, portanto, necessariamente par.

3.7 O que muda em grafos direcionados?

Em grafos direcionados, distinguimos grau de entrada $\deg^-(v)$ e grau de saída $\deg^+(v)$. Cada arco sai de um vértice e entra em outro. Por isso, a soma dos graus de saída é igual ao número de arcos, e a soma dos graus de entrada também é igual ao número de arcos.

$$\Sigma \deg^+(v) = |E| = \Sigma \deg^-(v)$$

Unidade 4

Planaridade dos Grafos - Isomorfismo - Teorema de Kuratowski

4.1 Isomorfismo: quando dois grafos têm a mesma estrutura

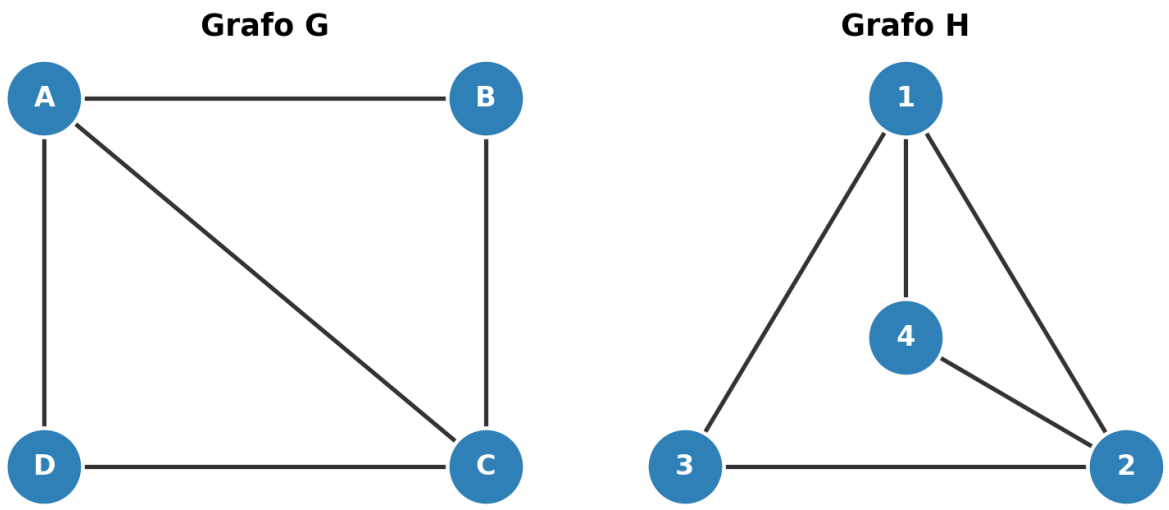
Dois grafos podem ser desenhados de maneiras completamente diferentes e ainda representar exatamente a mesma estrutura de conexões. O conceito de isomorfismo formaliza essa ideia: ele ignora o nome e a posição visual dos vértices e observa apenas quais pares estão conectados (BONDY; MURTY, 2008).

$$f : V(G) \to V(H) \text{ é uma bijeção que preserva adjacências}$$

$$\{u,v\} \in E(G) \Leftrightarrow \{f(u),f(v)\} \in E(H)$$

Pense assim

Um isomorfismo funciona como uma troca consistente de nomes. Se renomearmos os vértices de G segundo uma bijeção e todas as arestas continuarem correspondendo às arestas de H, os grafos são estruturalmente iguais.



Uma bijeção possível: A→1, B→3, C→2, D→4

Figura 7 — Dois desenhos distintos com a mesma estrutura de adjacências.

No exemplo, a correspondência A→1, B→3, C→2 e D→4 preserva todas as cinco arestas. Por isso, G e H são isomorfos. Essa verificação exige mais do que comparar quantidades: é necessário construir ou justificar uma bijeção que preserve a incidência das arestas.

4.2 Invariantes úteis no diagnóstico de isomorfismo

Antes de tentar construir uma bijeção, podemos comparar propriedades que todo isomorfismo obrigatoriamente preserva. Se uma delas for diferente, os grafos não podem ser isomorfos.

- Número de vértices $|V|$ e número de arestas $|E|$.

- Sequência de graus dos vértices, incluindo as multiplicidades de cada grau.
- Quantidade de componentes conexas e presença de vértices isolados.
- Existência e quantidade de estruturas específicas, como triângulos e ciclos de certos comprimentos.
- Conectividade e padrões de vizinhança que possam ser distinguidos estruturalmente.

Importante

Ter os mesmos valores de |V|, |E| e a mesma sequência de graus é necessário, mas não é suficiente. Dois grafos podem compartilhar esses invariantes e ainda não serem isomorfos. Os invariantes descartam candidatos; a bijeção que preserva adjacências é o critério decisivo.

4.3 Planaridade: propriedade do grafo, não do desenho

Um grafo é planar quando existe pelo menos uma forma de desenhá-lo no plano sem que arestas se cruzem, exceto nos próprios vértices. Um desenho específico pode apresentar cruzamentos mesmo quando o grafo é planar. Por isso, observar um cruzamento na figura não basta para concluir não planaridade.

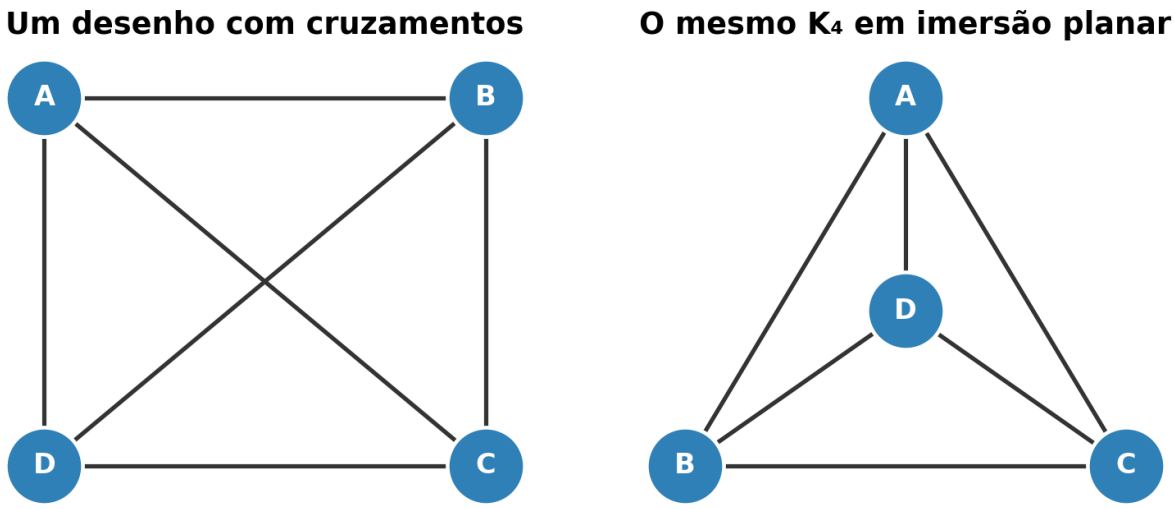


Figura 8 — O K_4 pode parecer "cruzado" e ainda assim admite uma representação planar.

Ponto de atenção

A pergunta correta não é "este desenho tem cruzamentos?", mas sim "existe algum desenho equivalente sem cruzamentos?". Planaridade é uma propriedade abstrata do grafo.

4.4 Fórmula de Euler para grafos planares

Quando um grafo planar conexo é desenhado sem cruzamentos, o plano fica dividido em regiões chamadas faces. A região externa também conta como uma face. Para qualquer representação planar conexa vale a relação:

$$|V| - |E| + |F| = 2$$

A fórmula de Euler permite verificar consequências importantes. Para grafos simples planares com $|V| \geq 3$, obtém-se a condição necessária $|E| \leq 3|V| - 6$. Se o grafo planar também for bipartido, não há ciclos de comprimento 3 e a condição pode ser refinada para $|E| \leq 2|V| - 4$ (WEST, 2001).

Situação	Condição necessária	Uso
Planar, simples, $\|V\| \geq 3$	$\|E\| \leq 3\|V\| - 6$	Descartar grafos muito densos
Planar e bipartido, $\|V\| \geq 3$	$\|E\| \leq 2\|V\| - 4$	Explorar ausência de triângulos
Planar e conexo	$\|V\| - \|E\| + \|F\| = 2$	Relacionar vértices, arestas e faces

Não confunda

As desigualdades acima fornecem condições necessárias, não uma caracterização completa. Satisfazer a desigualdade não garante planaridade; violá-la, entretanto, prova que o grafo não pode ser planar.

4.5 K_5, $K_{3,3}$ e o Teorema de Kuratowski

Dois grafos ocupam papel central no estudo da planaridade: K_5, o grafo completo com cinco vértices, e $K_{3,3}$, o grafo bipartido completo com três vértices em cada parte. Ambos são não planares.

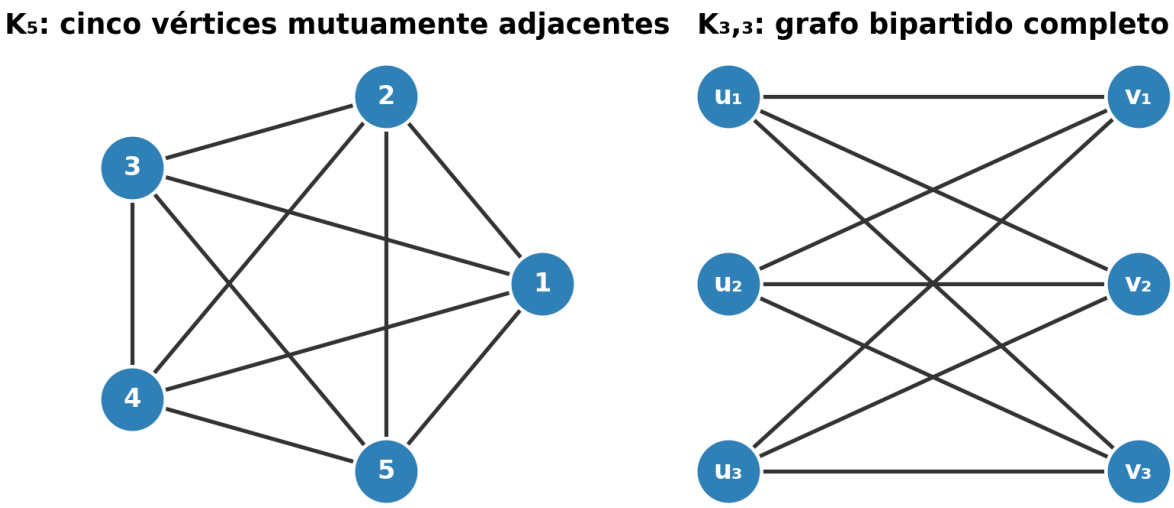


Figura 9 — Os dois padrões fundamentais de não planaridade no Teorema de Kuratowski.

O Teorema de Kuratowski afirma que um grafo finito é planar se, e somente se, não contém como subgrafo uma subdivisão de K_5 ou de $K_{3,3}$ (WEST, 2001; DIESTEL, 2017). Subdividir uma aresta significa substituí-la por um caminho, inserindo vértices de grau 2 ao longo dessa aresta. Assim, um grafo pode esconder um desses padrões mesmo sem possuir literalmente um K_5 ou um $K_{3,3}$ como subgrafo.

Leitura estrutural

Kuratowski transforma a pergunta geométrica “consigo desenhar sem cruzamentos?” em uma pergunta estrutural: “há dentro do grafo uma subdivisão de K_5 ou $K_{3,3}$?”. Essa mudança de perspectiva é típica da Teoria dos Grafos.

4.6 Exemplo de diagnóstico rápido

Considere K_5. Ele possui $|V|=5$ e $|E|=10$. Se fosse um grafo simples planar, deveria satisfazer $|E| \leq 3|V|-6$, isto é, $10 \leq 9$, o que é falso. Portanto, K_5 é não planar. Já $K_{3,3}$ possui 6 vértices e 9 arestas; a desigualdade geral não o elimina, mas, por ser bipartido, teríamos $9 \leq 2 \cdot 6 - 4 = 8$, novamente uma contradição.

Síntese da Unidade 4

Isomorfismo compara estruturas por meio de uma bijeção que preserva adjacências. Planaridade pergunta se existe um desenho sem cruzamentos. A fórmula de Euler e suas desigualdades oferecem testes necessários, enquanto o Teorema de Kuratowski caracteriza a planaridade pela ausência de subdivisões de K_5 e $K_{3,3}$.

Unidade 5

Matriz de adjacência e Lista de adjacência

5.1 Por que representar um grafo dentro do computador?

A definição matemática G=(V,E) descreve quais vértices e arestas existem, mas um algoritmo precisa de uma estrutura concreta para consultar vizinhos, verificar arestas e percorrer o grafo. Duas representações clássicas são a matriz de adjacência e a lista de adjacência (CORMEN et al., 2022).

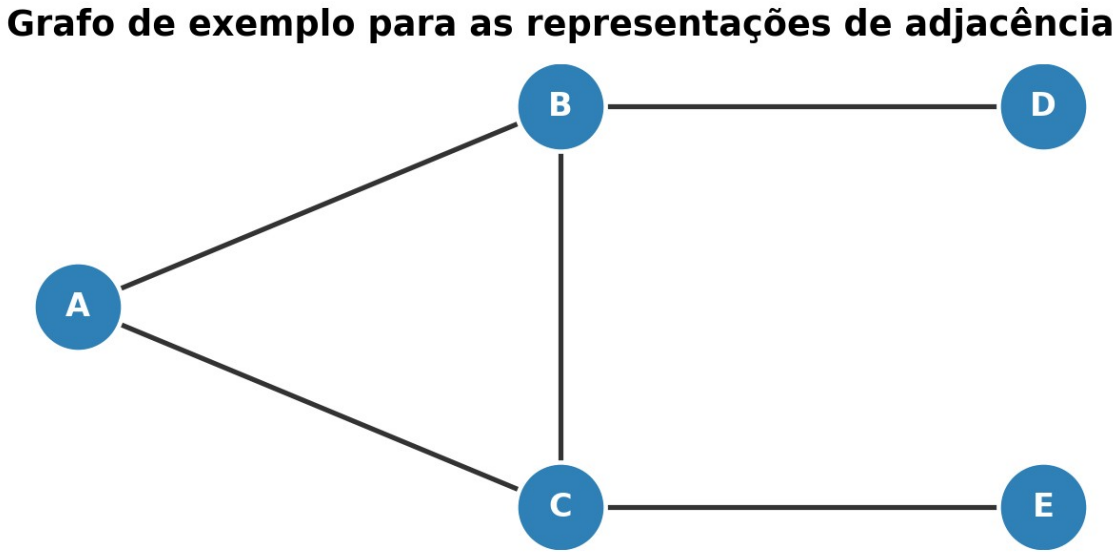


Figura 10 — Grafo simples utilizado para comparar matriz e lista de adjacência.

Para o grafo da Figura 10, considere a ordem dos vértices A, B, C, D e E. As arestas são {A,B}, {A,C}, {B,C}, {B,D} e {C,E}.

5.2 Matriz de adjacência

A matriz de adjacência de um grafo com n vértices é uma matriz A de dimensão n×n. Em um grafo simples não direcionado, a posição a_{ij} recebe 1 quando os vértices i e j são adjacentes e 0 quando não há aresta entre eles.

$$\mathbf{a_{ij} = 1 \text{ se } \{v_i, v_j\} \in E; \text{ caso contrário, } a_{ij} = 0}$$

	A	B	C	D	E
A	0	1	1	0	0
B	1	0	1	1	0
C	1	1	0	0	1
D	0	1	0	0	0
E	0	0	1	0	0

- Em um grafo simples não direcionado, a matriz é simétrica em relação à diagonal principal.
- A diagonal principal contém zeros porque não há laços.

- A soma dos valores de uma linha é o grau do vértice correspondente, quando o grafo é simples e não direcionado.
- Em grafos direcionados, a matriz pode deixar de ser simétrica: a_{ij} registra o arco que sai de i e chega a j.

Vantagem principal

A existência de uma aresta entre dois vértices pode ser consultada diretamente em uma posição da matriz. A desvantagem é reservar espaço para todos os n^2 pares possíveis, mesmo quando o grafo possui poucas arestas.

5.3 Lista de adjacência

Na lista de adjacência, cada vértice armazena apenas seus vizinhos. A mesma estrutura da Figura 10 pode ser representada da seguinte forma:

Vértice	Lista de vizinhos
A	B, C
B	A, C, D
C	A, B, E
D	B
E	C

Em um grafo não direcionado, cada aresta aparece duas vezes nas listas: uma vez na lista de cada extremidade. Por isso, o tamanho total das listas é proporcional a $2|E|$ e o consumo de memória é $O(|V|+|E|)$.

5.4 Comparando custo de memória e operações

Critério	Matriz de adjacência	Lista de adjacência
Memória	$O(\|V\|^2)$	$O(\|V\|+\|E\|)$
Testar se {u,v} existe	O(1)	O(deg(u)) em lista simples
Enumerar vizinhos de u	$O(\|V\|)$	O(deg(u))
Mais adequada para	Grafos densos ou consultas diretas	Grafos esparsos e percursos

Decisão de implementação

Não existe uma representação universalmente “melhor”. A escolha depende da densidade do grafo e das operações dominantes. Um sistema que verifica milhões de pares pode favorecer matriz; um algoritmo que percorre vizinhos em uma rede esparsa tende a favorecer listas.

5.5 Matrizes e listas em grafos ponderados

Quando as arestas possuem pesos, a estrutura precisa guardar também o valor associado a cada conexão. Em uma matriz, pode-se armazenar o peso em a_{ij} e reservar um valor especial para ausência de aresta. Em uma lista, cada vizinho pode ser armazenado junto com seu peso, por exemplo, no grafo da Figura 12, B → [(A,4), (C,2), (D,1), (E,7)].

Ponto de atenção

Usar 0 para representar “sem aresta” pode ser inadequado quando peso zero é permitido. A implementação deve distinguir claramente ausência de conexão de uma aresta cujo peso realmente vale zero.

Síntese da Unidade 5

A matriz de adjacência oferece acesso direto aos pares de vértices, ao custo de $O(|V|^2)$ de memória. A lista de adjacência armazena somente as conexões existentes e ocupa $O(|V|+|E|)$, sendo especialmente eficiente em grafos esparsos e em algoritmos de percurso.

Unidade 6

Percurso em Profundidade (DFS) e Percurso em Largura (BFS)

6.1 O problema de percorrer um grafo

Percorrer um grafo significa visitar seus vértices de forma sistemática, evitando processar indefinidamente as mesmas conexões. BFS e DFS resolvem esse problema com estratégias distintas. Ambas mantêm um conjunto de vértices visitados; o que muda é a ordem em que os próximos vértices são escolhidos (CORMEN et al., 2022).

Ideia central

BFS explora o grafo por camadas, aproximando-se primeiro dos vértices mais próximos da origem. DFS segue um caminho o mais profundamente possível antes de voltar e explorar alternativas.

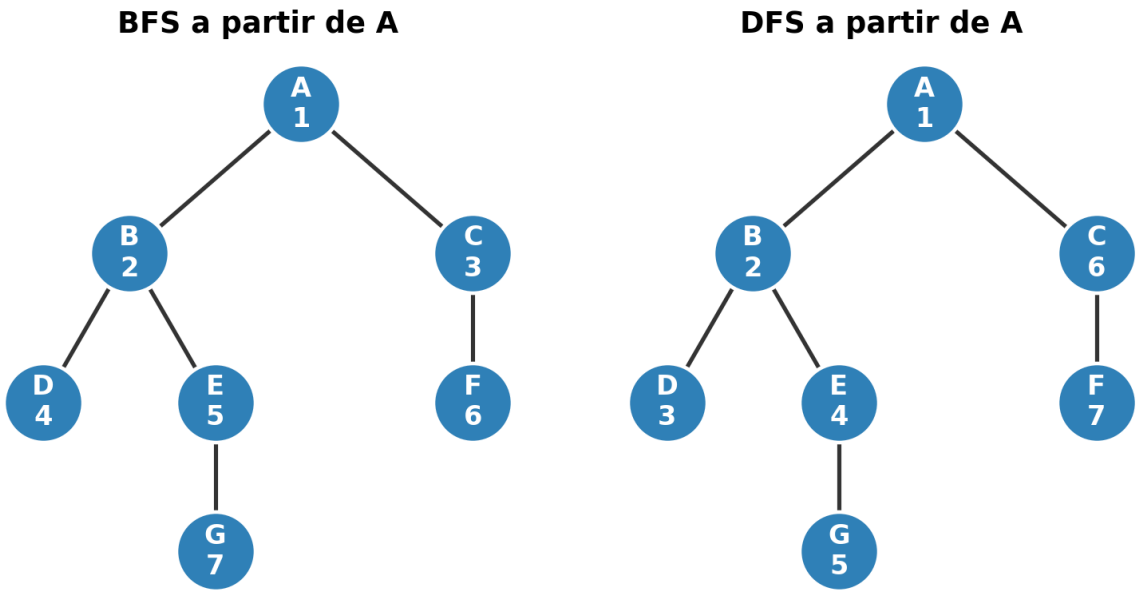


Figura 11 — Ordens possíveis de visita por BFS e DFS a partir de A.

Ordem de vizinhos

A ordem exata de visita pode variar. Na Figura 11 assumimos que, sempre que houver escolha, os vizinhos são considerados em ordem alfabética. Outra ordenação da lista de adjacência pode produzir outra sequência válida.

6.2 BFS — Busca ou Percurso em Largura

A BFS utiliza uma fila (FIFO). O primeiro vértice descoberto é o primeiro a ser expandido. A partir da origem,

ela descobre todos os vértices a distância 1, depois todos a distância 2 e assim sucessivamente.

1. Marque o vértice inicial como visitado e coloque-o na fila.
2. Retire o primeiro vértice da fila e examine seus vizinhos.
3. Para cada vizinho ainda não visitado, marque-o e coloque-o no fim da fila.
4. Repita até que a fila fique vazia.

No grafo da Figura 11, partindo de A, uma BFS com vizinhos em ordem alfabética visita A, B, C, D, E, F, G. Os níveis são: {A}, depois {B,C}, depois {D,E,F} e, por fim, {G}.

Aplicação importante

Em grafos não ponderados, a BFS calcula caminhos mínimos em número de arestas a partir da origem. O primeiro momento em que um vértice é descoberto já determina sua menor distância em quantidade de arestas.

6.3 DFS — Busca ou Percurso em Profundidade

A DFS pode ser implementada com uma pilha explícita ou com chamadas recursivas. Ela escolhe um vizinho ainda não visitado e continua avançando. Quando chega a um vértice sem vizinhos novos, retorna ao ponto anterior e tenta outra alternativa.

1. Visite o vértice atual.
2. Escolha um vizinho ainda não visitado.
3. Aplique o mesmo procedimento a esse vizinho.
4. Quando não houver vizinhos novos, faça o retorno (backtracking).

Na Figura 11, considerando a ordem alfabética, uma DFS possível é A, B, D, E, G, C, F. O percurso “mergulha” pelo ramo iniciado em B antes de retornar a A para explorar C.

6.4 O que cada percurso ajuda a resolver?

Tarefa	BFS	DFS
Menor caminho em grafo não ponderado	Muito apropriada	Não garante o menor caminho
Exploração por níveis/distância	Natural	Não é o foco
Detecção de componentes	Sim	Sim

Tarefa	BFS	DFS
Backtracking e exploração profunda	Menos natural	Natural
Ordenação topológica em DAG	Não na forma básica	Aplicação clássica

6.5 Complexidade

Com lista de adjacência, cada vértice é marcado no máximo uma vez e cada aresta é examinada um número constante de vezes. Assim, tanto BFS quanto DFS executam em $O(|V|+|E|)$. Com matriz de adjacência, o custo típico de examinar vizinhos passa a $O(|V|^2)$.

Rigor metodológico

A complexidade do algoritmo depende também da representação escolhida para o grafo. Dizer apenas “BFS é O(V+E)” pressupõe uma estrutura que permita enumerar somente os vizinhos existentes, como uma lista de adjacência.

Síntese da Unidade 6

BFS usa fila e explora por camadas, sendo fundamental para menores caminhos em grafos não ponderados. DFS usa pilha ou recursão e aprofunda caminhos antes de voltar. Com listas de adjacência, ambas percorrem o grafo em $O(|V|+|E|)$.

Unidade 7

Algoritmo de Dijkstra e Bellman-Ford

7.1 Caminhos mínimos em grafos ponderados

Quando as arestas possuem pesos, contar apenas o número de arestas deixa de ser suficiente. O custo de um caminho é a soma dos pesos de suas arestas. O problema de caminho mínimo de fonte única procura, para uma origem s, a menor distância possível até cada vértice alcançável (CORMEN et al., 2022).

dist(s,v) = menor soma de pesos entre s e v

Relaxamento

Dijkstra e Bellman-Ford compartilham uma operação central: testar se passar por um vértice u melhora a distância conhecida de v. Para uma aresta (u,v) de peso w, tentamos substituir d[v] por d[u]+w quando esse valor for menor.

7.2 Algoritmo de Dijkstra

Dijkstra é um algoritmo guloso para grafos cujos pesos de aresta são não negativos. Ele mantém uma estimativa de distância para cada vértice e, a cada etapa, escolhe o vértice ainda não finalizado com a menor distância conhecida.

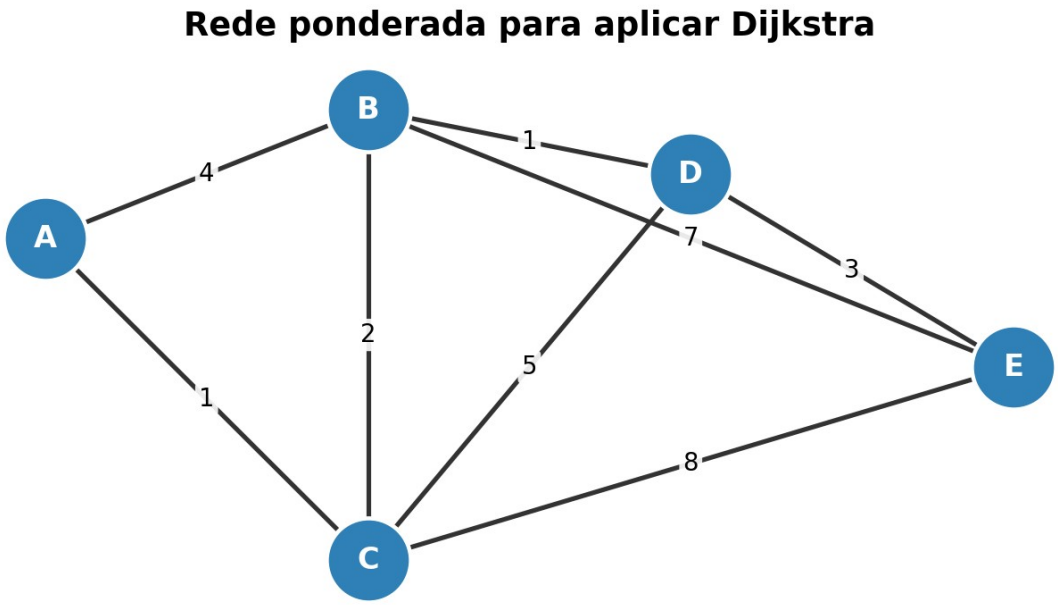


Figura 12 — Grafo ponderado para um exemplo de Dijkstra a partir de A.

1. Inicialize d[A]=0 e d[v]=∞ para os demais vértices.

2. Escolha o vértice não finalizado com menor d.

3. Relaxe todas as arestas que saem desse vértice.

4. Finalize o vértice escolhido e repita até não haver vértices alcançáveis pendentes.

Vértice	Distância mínima a partir de A	Um caminho mínimo
A	0	A
C	1	A–C
B	3	A–C–B
D	4	A–C–B–D
E	7	A–C–B–D–E

Por exemplo, C recebe distância 1 diretamente de A. Depois, relaxar C melhora B de 4 para 3. Em seguida, B melhora D para 4; por fim, D melhora E para 7.

Restrição fundamental

Dijkstra não deve ser usado quando há arestas de peso negativo. A estratégia gulosa depende da garantia de que, depois de escolher o menor rótulo disponível, nenhuma extensão futura com pesos não negativos poderá torná-lo menor.

7.3 Algoritmo de Bellman-Ford

Bellman-Ford também resolve o problema de caminho mínimo de fonte única, mas aceita arestas com pesos negativos. Em vez de “finalizar” vértices por uma escolha gulosa, ele relaxa repetidamente todas as arestas do grafo.

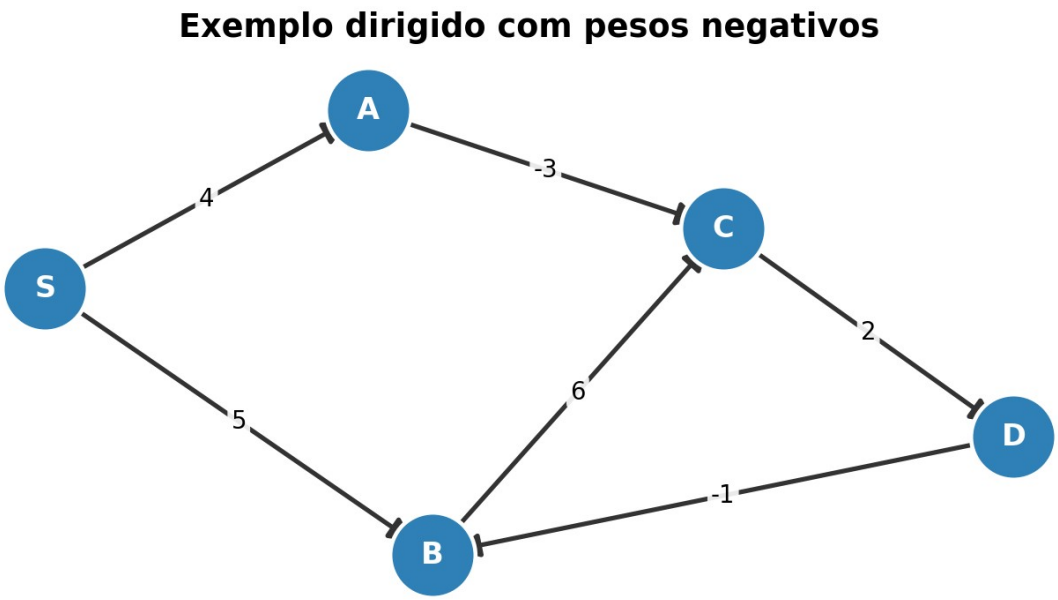


Figura 13 — Grafo dirigido em que Bellman-Ford pode aproveitar uma aresta de peso negativo.

Em um caminho simples, podem existir no máximo $|V|-1$ arestas. Por isso, após $|V|-1$ rodadas completas de relaxamento, todas as menores distâncias estão determinadas, desde que não exista ciclo negativo alcançável a partir da fonte.

1. Inicialize a origem com distância 0 e os demais vértices com ∞.
2. Repita $|V|-1$ vezes: percorra todas as arestas e aplique o relaxamento.
3. Faça uma rodada adicional sobre as arestas.
4. Se alguma distância ainda puder diminuir, existe um ciclo de peso negativo alcançável a partir da origem.

Na Figura 13, a partir de S: d[A] = 4 e d[B] = 5; o arco A→C, de peso negativo, dá d[C] = 4 + (−3) = 1; em seguida, C→D dá d[D] = 3; e D→B melhora d[B] de 5 para 3 + (−1) = 2. O ciclo B→C→D→B tem peso 6 + 2 − 1 = 7, que não é negativo; portanto, as distâncias finais são S = 0, A = 4, B = 2, C = 1 e D = 3.

Por que detectar ciclo negativo?

Se um ciclo alcançável possui soma de pesos negativa, podemos percorrê-lo repetidamente e reduzir o custo do caminho sem limite. Nesse caso, não existe uma distância mínima finita para os vértices afetados.

7.4 Dijkstra ou Bellman-Ford?

Critério	Dijkstra	Bellman-Ford
Pesos negativos	Não admite	Admite
Detecta ciclo negativo	Não	Sim, se alcançável da fonte
Estratégia	Gulosa com menor distância	Relaxamento repetido de todas as arestas
Complexidade típica	O((V+E) log V) com heap	O(VE)
Uso preferencial	Pesos não negativos e desempenho	Pesos negativos ou diagnóstico de ciclos

Decisão algorítmica

A presença ou ausência de pesos negativos não é um detalhe de implementação: ela altera qual algoritmo é matematicamente correto. Escolher Dijkstra apenas porque “é mais rápido” pode produzir resultados incorretos quando a hipótese de não negatividade não é satisfeita.

Síntese da Unidade 7

Dijkstra usa uma estratégia gulosa eficiente, mas exige pesos não negativos. Bellman-Ford é mais custoso, porém aceita pesos negativos e detecta ciclos negativos alcançáveis. Ambos são fundamentados na operação de relaxamento das arestas.

Unidade 8

Algoritmo de Prim

8.1 Antes de Prim: árvore geradora mínima

Considere um grafo não direcionado, conexo e ponderado. Uma árvore geradora é um subgrafo que contém todos os vértices, permanece conexo e não possui ciclos. Como toda árvore com $|V|$ vértices possui exatamente $|V|-1$ arestas, uma árvore geradora elimina conexões redundantes sem desconectar a rede.

Uma árvore geradora mínima (AGM ou MST, de Minimum Spanning Tree) é uma árvore geradora cuja soma dos pesos das arestas é a menor possível entre todas as árvores geradoras do grafo (CORMEN et al., 2022).

$$\textbf{peso}(T) = \Sigma_{e \in T}\, w(e) \quad \textbf{e} \quad |E(T)| = |V|-1$$

Exemplo de aplicação

Se vértices representam prédios e pesos representam o custo de instalar cabos entre pares de prédios, uma AGM conecta todos os locais com o menor custo total, sem exigir caminhos mínimos individuais a partir de um prédio específico.

8.2 Ideia do algoritmo de Prim

Prim constrói uma única árvore gradualmente. Começa por qualquer vértice e mantém um conjunto S com os vértices já incorporados. Em cada etapa, escolhe a aresta de menor peso que cruza o corte entre S e V−S, isto é, uma aresta com uma extremidade dentro da árvore e outra fora dela.

1. Escolha um vértice inicial e coloque-o na árvore.
2. Entre todas as arestas que ligam a árvore a um vértice externo, escolha a de menor peso.
3. Adicione essa aresta e o novo vértice à árvore.
4. Repita até que todos os vértices estejam incluídos.

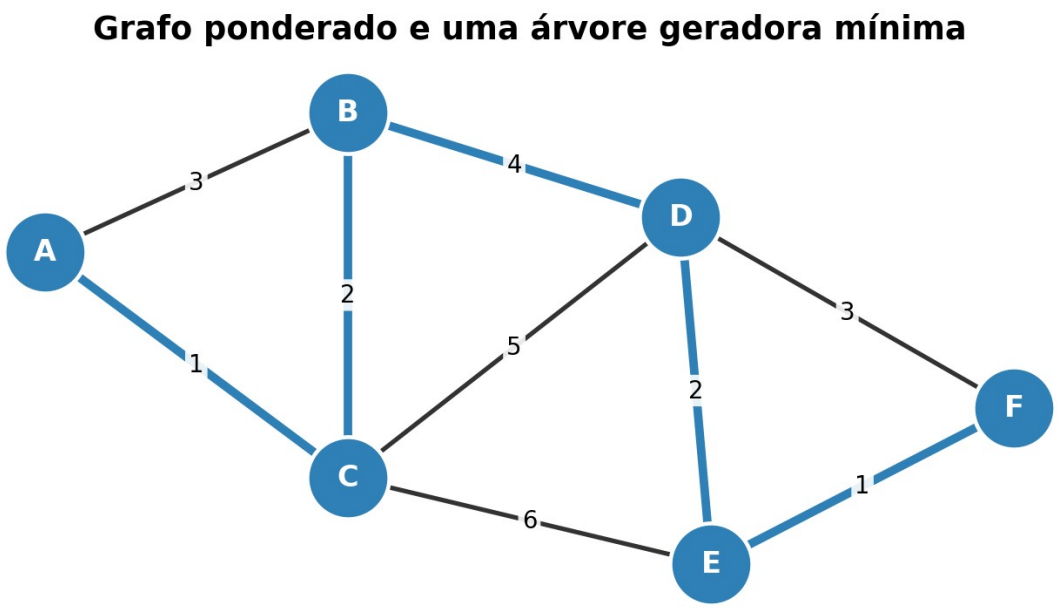


Figura 14 — Exemplo de árvore geradora mínima obtida no grafo ponderado.

8.3 Exemplo passo a passo

Aplicando Prim a partir de A no grafo da Figura 14, uma sequência possível é:

Etapa	Aresta escolhida	Peso	Vértices na árvore
1	A–C	1	A, C
2	C–B	2	A, B, C
3	B–D	4	A, B, C, D
4	D–E	2	A, B, C, D, E
5	E–F	1	A, B, C, D, E, F

Peso total = 1 + 2 + 4 + 2 + 1 = 10

A sequência pode mudar quando há empates, e um grafo pode possuir mais de uma AGM. O que deve permanecer mínimo é o peso total. A escolha é justificada pela propriedade do corte: a aresta de menor peso que cruza o corte entre a árvore e o restante do grafo é segura, isto é, sempre existe uma AGM que a contém (CORMEN et al., 2022).

8.4 Prim não é Dijkstra

Os dois algoritmos podem usar uma fila de prioridade e ambos escolhem repetidamente algum valor mínimo, o que torna fácil confundi-los. Entretanto, seus objetivos e significados são diferentes.

Aspecto	Prim	Dijkstra
Objetivo	Minimizar o peso total da árvore	Minimizar a distância desde uma fonte
Estrutura produzida	Árvore geradora mínima	Árvore de caminhos mínimos
Chave de um vértice	Menor aresta que o conecta à árvore	Menor distância conhecida desde a fonte
Pesos negativos	Podem existir em MST	Não são permitidos no algoritmo clássico

Não confunda

Uma AGM não garante que o caminho entre dois vértices dentro da árvore seja o caminho de menor custo existente no grafo original. O objetivo de Prim é minimizar a soma global das arestas selecionadas, não as distâncias individuais a partir de uma origem.

8.5 Complexidade e condição de conectividade

Com lista de adjacência e fila de prioridade baseada em heap binário, Prim pode ser implementado em $O(|E| \log |V|)$. Uma implementação simples sobre matriz de adjacência costuma executar em $O(|V|^2)$, sendo adequada para grafos densos ou de tamanho moderado.

Se o grafo for desconexo, não existe uma única árvore geradora que alcance todos os vértices. Nesse caso, executar o procedimento em cada componente produz uma floresta geradora mínima, com uma árvore para cada componente conexa.

Síntese da Unidade 8

Prim constrói uma árvore geradora mínima adicionando repetidamente a aresta de menor peso que conecta a árvore atual a um novo vértice. O objetivo é minimizar o custo total de conexão da rede, e não calcular caminhos mínimos a partir de uma fonte.

REFERÊNCIAS

BONDY, J. A.; MURTY, U. S. R. Graph Theory. New York: Springer, 2008.

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. Introduction to Algorithms. 4. ed. Cambridge, MA: MIT Press, 2022.

DIESTEL, Reinhard. Graph Theory. 5. ed. Berlin: Springer, 2017.

ROSEN, Kenneth H. Discrete Mathematics and Its Applications. 8. ed. New York: McGraw-Hill, 2019.

WEST, Douglas B. Introduction to Graph Theory. 2. ed. Upper Saddle River: Prentice Hall, 2001.

Declaração de uso de IA: este material foi elaborado com apoio de ferramenta de IA generativa na redação, nos exemplos e na revisão. O conteúdo foi conferido pelo grupo, com exemplos recalculados e referências verificadas, e as decisões finais são dos integrantes, conforme o Guia de Boas Práticas no Uso de IA da USCS.